

ADVANCED DATA MANAGEMENT 2025/2026
June 10, 2026
CLOSED BOOK

FIRST NAME: BUSKRA
SID: 6180175

LAST NAME: MUNEER

Q1. (up to 10 lines) [only for COMPUTER ENGINEERING STUDENTS]
Describe and briefly compare block-based, hash-based, and range-based partitioning approaches.

Q2. (up to 10 lines)

With reference to the three-step design methodology and Cassandra, explain why choosing aggregation entities among the selection entities is considered a good heuristic.

The aggregation entities must be among the selection entities is considered a good heuristic because we access the node based on that entity to have an efficient retrieval or lookup and to avoid accessing all nodes, because in Cassandra we don't have joins and we create separate tables and include the selection attributes in primary key to access and basically to have the nodes stored in a sequential manner.

NO

HUNTER

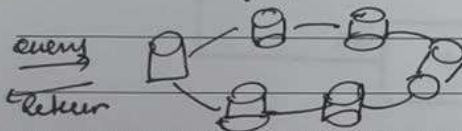
Q3. (up to 10 lines)

Discuss the main challenges in partitioning graph-based data and the solutions that can be applied to address them.

1.5
(2?)
Main challenge in graph based partitioning: if nodes are distributed across border it will be hard to access or gain retrieval to the particular required node and the relationship is broken.

Solution: → FABRIC NEO4j →

I can access or query to one server which will find the relevant graph and return that one.



Q4. (up to 10 lines) [only for COMPUTER SCIENCE STUDENTS]

You are building a hybrid search system with vector similarity and metadata filter: language = "Italian". Explain the risks of naive post-filtering and propose one strategy to mitigate false negatives.

1.5
(2)
The risk of post-rative filtering is that the system will already be having K records in retrieval, applying a post filter could result in K-m or 0 empty results.

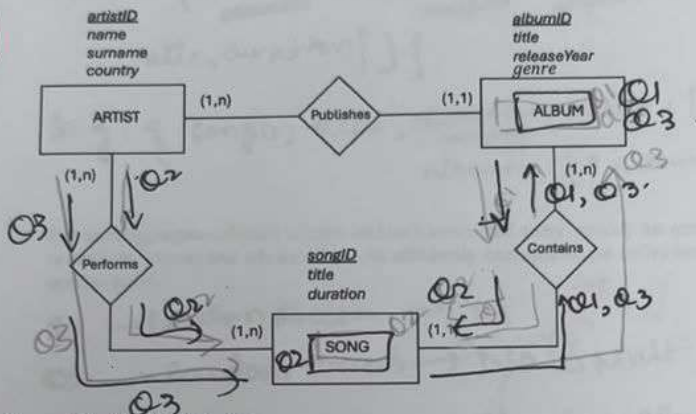
The synthetic strategy to mitigate the retrieved false negatives to include all retrievals along with filter.

too much

HUMMER

Exercise 1 [NoSQL aggregate-oriented data stores]

Consider the following entity relationship diagram, which describes artists, the songs they perform, and the albums they publish. The notation is simplified: entity attributes are written in *italic*, identifiers are underlined.



The workload is the following:

- **Query 1:** Retrieve all albums of a given genre and the songs contained in each album.
- **Query 2:** Retrieve all songs with a given duration, together with the corresponding artists and album.
- **Query 3:** Retrieve all albums released in a given year, together with the corresponding artist and songs.

Based on the previous ER schema and workload, answer the following questions, **motivating all your choices**.

1. Use the aggregate-oriented design methodology (STEP 1-2-3) to design a set of logical aggregates, in meta-notation, for the given domain and the selected workload. Present the output of each step of the methodology.

Queries

- Q1 (Album [Album (albumID, genre) - !], [Songs (songID, ~~title~~, duration) - C])
- ✓ Q2 (Song, [Song (duration) - !], [Artist (artistID) - P, Album (albumID) - C])
- Q3 (Album, [Album (releaseYear) - !], [Artist (artistID) - P, Song (songID) - C])

Annotation: please annotate the input ER diagram

Aggregates

Album: { albumID, title, releaseYear, genre, artistID, song: [{ songID, title, duration }] } . ok

Song: { songID, title, duration, artistID, album: [{ albumID }] }

HANY

Partition Key

① → genre, releaseyear, albumID
 ② → songID, title, duration

⊕

MUNEEB

wh

2. Revise and extend the logical aggregate schemas to consider partition keys and super-identifier information.

Album: { albumID, releaseyear, genre, artistID, song: [{ songID, title, duration }] }

Song: { songID, title, duration, artist: [{ artistID }], album: [{ albumID }] }

different

3. For each aggregate-oriented schema and each associated query, specify the query type (either batch or random-access) and whether it can be efficiently executed on the collection (for random-access queries only).

Q1 → Random Access → Yes

Q2 → Random Access → No → Multiple nodes accessed

Q3 → Random Access → No → Multiple nodes accessed

Efficient → if one or less nodes, sequentially accessed.

4. Determine which indexes (if any) should be created to improve the efficiency of workload queries.

Index → Q2 → artistID, albumID ⇒ will lead to particular targeted nodes.

Index → Q3 → artistID, songID

why?

5. Suppose to implement the designed aggregates in MongoDB.

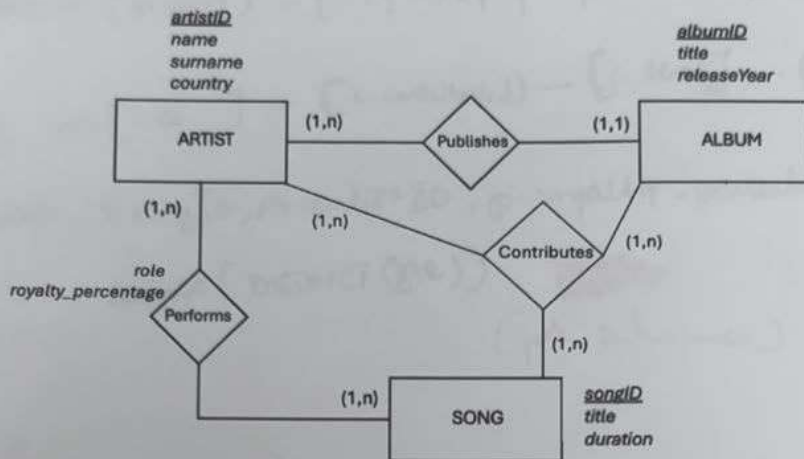
- Specify which indexes should be created and which of them should be unique.
- Specify Query 3 in the MongoDB interaction language.

Indexes in MongoDB ⇒ Hashed/unique.

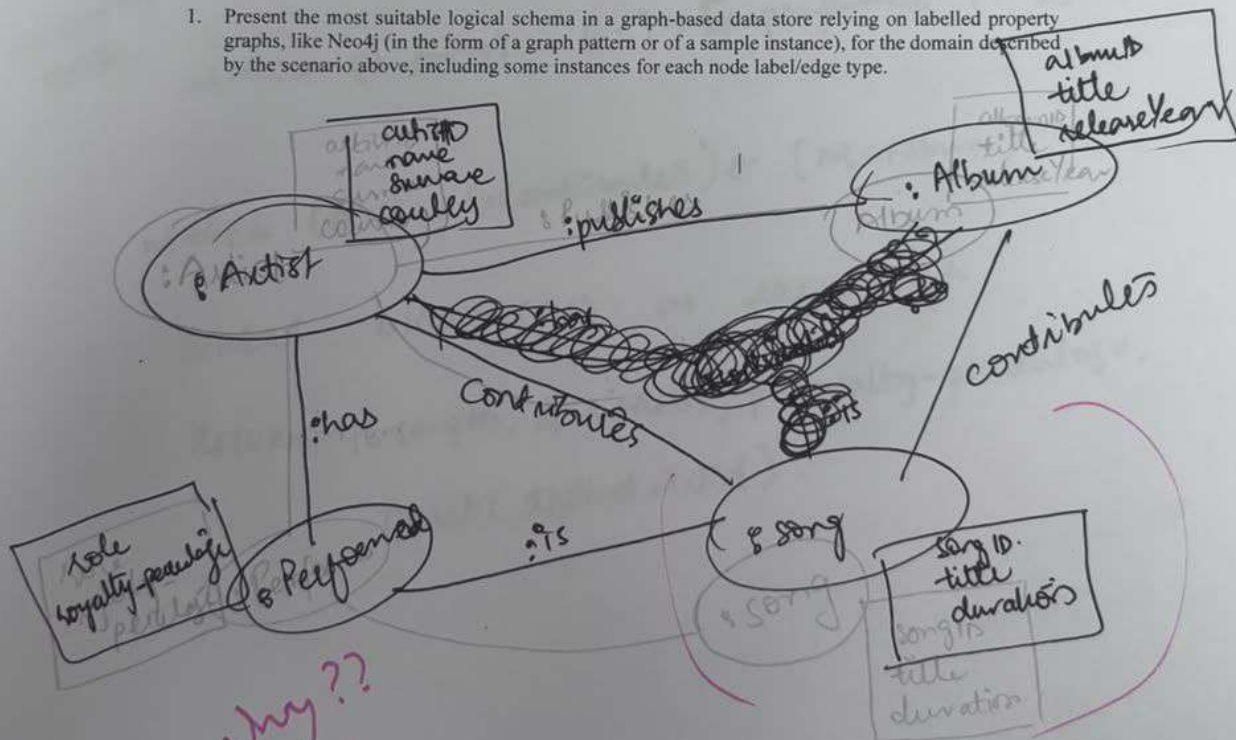
MUNEER

Exercise 2 [NoSQL graph-based data stores]

Consider the following entity relationship diagram, which describes artists, the songs they perform, and the albums they publish. The notation is simplified: entity attributes are written in *italic*, identifiers are underlined.



1. Present the most suitable logical schema in a graph-based data store relying on labelled property graphs, like Neo4j (in the form of a graph pattern or of a sample instance), for the domain described by the scenario above, including some instances for each node label/edge type.



TWNEER

2. Assuming the information above is represented using a labelled property graph (Neo4j), write a Cypher query that, for each song that is performed by an artist, returns the song ID, the artist ID, the royalty percentage, and the number of distinct albums to which the song is contributed by that artist.

Approach
X 1).

MATCH (s:song) - [:is] -> (p:performed) - [:by] -> (A:Artist)
(Al:Album) <- [:to] - (c:contributed) - [:that] - (A:Artist)
RETURN s.songID, A.artistID, p.royalty-percentage,
count(DISTINCT Al)).

not existing

(pos. schema)

0,5

Approach
2
Group by
filter.

MATCH (s:song) - [:is] -> (p:performed) - [:by] -> (A:Artist)

MATCH (A) -> (c:contributed) -> (Al:Album)

WHERE DISTINCT (Al) as distinctArtist

RETURN s.songID, A.artistID, p.royalty-percentage,
count(distinct Artist).

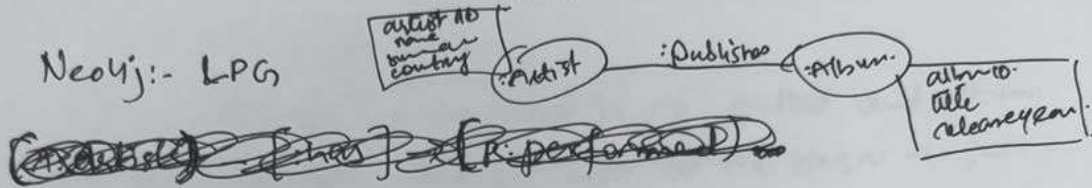
KUNDEER

Exercise 3 [Data management for AI driven application] [only for COMPUTER SCIENCE STUDENTS]

With reference to scenario discussed in Exercise 2.1 for LPG:

1. Consider one small example instance of your choice (containing at least an instance of the relationships) and discuss the differences in how the relationships are represented in a labelled property graph (e.g., Neo4j) and in RDF. Your answer should show the graphs, clearly explain and compare the modeling choices adopted in each case.

Neo4j:- LPG



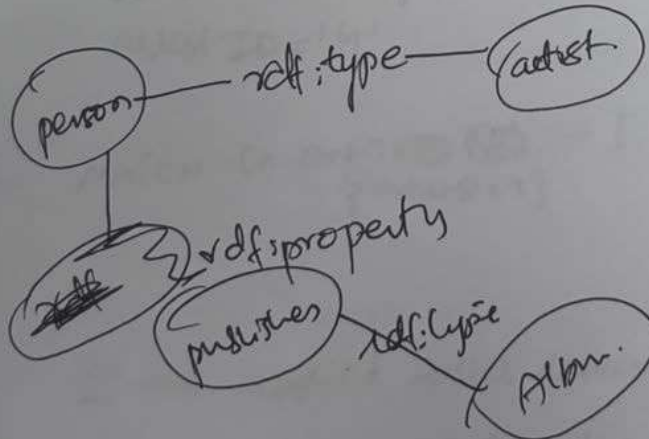
$(A:Artist) - [:publishes] \rightarrow (A:Album)$

RDF

ex: ~~artist~~ person rdf:type ex:artist

ex: publishes rdf:property. ex: ~~album~~ album

ex: album rdf:type ex:album



The adopted choice is Neo4j because RDF solely state only the facts and needs extra resources to define relationships.

MODELING NOT A DIFFERENCE

just syntax

0.5 (0?)

FLUMER

?? Incomprehensible

2. Assume now that the information of the scenario above is represented as a graph (e.g., a labelled property graph or an RDF graph). Discuss whether and how such a representation could be exploited in a Retrieval-Augmented Generation (RAG) setting involving Large Language Models (LLMs). In particular,
- (a) Describe what data exists in your domain, what is structured vs unstructured. Specifically, in addition to the graph-structured data above specify what objects are embedded, chunking strategy, metadata schema. You may use diagrams, tables, or pseudo-schema definitions.

Assuming the need:

objects to be embedded is the Artist and Albums.
the chunking strategy will be the HNSW to get atleast k nearest neighbors and including the artist-ID and album ID as fillers in metadata schema.

- (b) Show an example of request that exploits both the Graph and the RAG component of the architecture. Highlight the parts that can be executed exactly and those that will be approximated.

Assuming a request :-

'2025' release year album is published by
artist-ID = '4'

RAG.

Graph: MATCH (A:Artist {ArtistID=4}) - [:published] -> (A:Album {release year=2025})

If the applied filter return the data that is True positive meaning the executed exactly. (IVF)
because in HNSW it will approximate to speed up which will compromise the accuracy.